ECE 498 BH — LLM Reasoning for Engineering

Assignment 3

Due 11:59 pm, 04/09/2026

Prachit Gupta `prachit2@illinois.edu`

Objective

In this assignment I extend the single-problem UAV waypoint-generation pipeline from Assignment 2 along the two requested axes: problem scaling and tool integration. The engineering field remains **robotics, specifically local motion planning for autonomous quadrotor UAVs**. I designed five related planning problems with increasing structural difficulty, implemented a single verifier framework that dispatches across all five, and then augmented the pipeline with a pre-defined planning tool that reuses the repository's existing global planning utilities.

Execution note. In the current codebase, the `-mode` flag controls whether the pipeline runs in `live` or `live` mode, the code queries the OpenAI API using a **fixed system prompt** from `fixed_prompt.txt` plus a **problem-specific user prompt** from `problem_prompts/P1.txt` through `problem_prompts/P5.txt`. In `mock` mode, the code does *not* query the model; it uses deterministic **hand-crafted** pass/fail cases so the whole evaluation stack is reproducible offline. Additionally, *pipeline_ common.py* contains the neccessary function defination that common to each file required to be submitted such as loading prompt, querying the model, and loading the problem manifest.

Submission note. Submitted files include : `verifier.py`, `baseline.py`, `tool_pipeline.py`, `tool_eval.py`, `refinement.py`, the hybrid A* + MPC-style tool, and `SETUP.md` as well as `fixed_prompt.txt`, `problem specific prompt files` and a link to a github repository for a larger project code-base from which I am using a global optimizer having more context than llm regarding the environment

1 Five-Problem Suite [15 pts]

Instructions: Design five problems in the **same** engineering field you used in Assignment 2 (or a new field, provided you state it clearly). The problems must span a range of difficulty *as measured by LLM pass rate* on a baseline pipeline (no tools, repeated sampling via five independent trials).

Problem	Difficulty Label	Target Baseline Pass Rate
P1	Easy	3–5 / 5
P2	Easy	3–5 / 5
P3	Hard	0–2 / 5
P4	Hard	0–2 / 5
P5	Hard	0 / 5

Engineering Field:

Robotics — motion planning for UAVs using short horizon waypoint generation in NED(North East-Down) coordinates under obstacle constraints.

Problem P1 (Easy):

Title: Clear Corridor Cruise
Problem statement. The quadrotor starts at NED (0.0, 0.0, −2.5) and must move toward the goal (8.0, 0.5, −2.5). Only two obstacles exist in the local scene: a cylinder centered at (4.3, 2.7) with radius 0.7 m and a box centered at (5.6, −2.8) with footprint 1.2×1.2 m. Both obstacles are far enough from the direct corridor that the main challenge is simply producing a clean and properly spaced local horizon.
Specifications and success criteria.

- Return exactly five (x, y) waypoint pairs in NED.
- Use one integer `selected_waypoint_index` in $[0, 4]$.
- Keep the first leg within 3.0 m of the current pose.
- Maintain at least 0.8 m clearance from the listed obstacles.
- The selected waypoint and final waypoint must both reduce goal distance.

Difficulty rationale. This is intentionally close to a straight-line planning case. A baseline LLM should often succeed because the correct behavior is simple: local, smooth, and nearly centered motion.

Problem P2 (Easy):

Title: Single-Obstacle Go-Around
Problem statement. The quadrotor starts at (0.0, 0.0, −2.5) and must move toward (10.0, 0.0, −2.5). A central cylinder at (3.6, 0.0) with radius 1.0 m blocks the straight route,

and two side guards further downrange at (6.0, 2.4) and (6.1, −2.5) limit how aggressively the vehicle can swing wide. The planner must commit to one side of the blocker and then bend back toward the center.

Specifications and success criteria. Same schema as P1, with 0.8 m obstacle clearance and strict local waypoint spacing. The local horizon must already show the go-around maneuver rather than aiming directly through the blocker.

Difficulty rationale. This is still easy because it only requires one meaningful routing choice, but it is harder than P1 because a naive straight-line answer immediately fails safety.

Problem P3 (Hard):

Title: Alternating Slalom Corridor

Problem statement. The quadrotor starts at (0.0, 0.0, −2.5) with a small forward velocity and must head toward (14.0, 0.2, −2.5) through a lane bounded by two long walls and four alternating cylinders. The slalom obstacles appear at approximately (3.0, −1.1), (5.4, 1.0), (7.9, −0.9), and (10.4, 1.1). A valid local horizon must already set up multiple future turns instead of only reacting to the nearest cylinder.

Specifications and success criteria. Same structured output as before, but with a tighter 0.85 m clearance requirement. Dynamic smoothness matters more here because the local horizon must alternate lateral motion without creating sharp reversals.

Difficulty rationale. This problem becomes hard because the planner has to reason over a sequence of turns rather than a single obstacle. Short-horizon textual reasoning often collapses into either overly greedy goal-seeking or oscillatory waypoint placement.

Problem P4 (Hard):

Title: Trap Exit Around a U-Shaped Blocker

Problem statement. The quadrotor starts at (0.0, 0.0, −2.5) and must reach (13.0, 0.0, −2.5). A center wall at (4.8, 0.0) plus upper and lower arms at (7.0, 1.8) and (7.0, −1.8) form a U-shaped trap, with an additional lower decoy obstacle at (8.8, −3.0). The direct route is blocked. The safe local plan must first bias upward, leave the trap through the upper opening, and avoid the tempting but blocked lower pocket.

Specifications and success criteria. Same output schema as P1, but with a 0.9 m clearance requirement. The final waypoint does not need to solve the entire map, but it must clearly begin the trap-exit maneuver.

Difficulty rationale. This is hard because the first safe motion can look less "goal-directed" than an unsafe greedy move. LLMs that optimize only the immediate heading toward the goal tend to run into the blocked interior.

Problem P5 (Hard):

Title: Assignment 2 Long-Horizon Cluttered Corridor

Problem statement. This is the final and hardest problem, and it is the one intentionally aligned with Assignment 2. The quadrotor begins at (−0.14, 0.31, −2.5) and must move toward (35.0, 3.0, −2.5) inside a long corridor bounded by lane walls and populated by eight obstacle clusters. The challenge is not just local collision avoidance but producing a five-waypoint horizon that commits to a globally sensible corridor line while still respecting local kinematics.

Specifications and success criteria. The output object must exactly match the Assignment 2 interface:

- `waypoints`: exactly five NED waypoint pairs,
- `selected_waypoint_index`: integer in $[0, 4]$,
- `reasoning`: short explanation.

The verifier checks format, step-length consistency, goal progress, and 0.9 m signed clearance against the explicit obstacle map.

Difficulty rationale. This is the hardest problem because the local horizon has to be globally informed. It is very easy for a baseline model to produce either overlong jumps, zero useful selected progress, or a path that cuts through one of the corridor obstacles.

2 Verifiers [25 pts]

Instructions: Extend the verifier framework from Assignment 2 to cover all five problems.

Verifier Code:

I implemented a unified verifier in `verifier.py`. It accepts a structured `WaypointSolution` Pydantic object, selects the correct `ProblemSpec`, and returns a dictionary with `"pass"`, `"details"`, and `"reason"`.

The verifier evaluates four categories for every problem:

1. **Format**: exactly 5 finite waypoints, legal selected index, non-empty reasoning.
2. **Kinematics**: first leg bound, segment-length bound, and acceleration-change proxy.
3. **Goal progress**: selected waypoint and final waypoint must both reduce distance to goal.
4. **Safety**: signed clearance and collision check against the explicit obstacle list.

Verifier snippet:

```python
def verify(solution: Any, problem_id: str | None = None) -> dict[str, Any]:
    problem = get_problem(problem_id or _CURRENT_PROBLEM_ID)
    manifest = _problem_manifest(problem.problem_id)

    fmt = _check_format(solution)
    if not fmt["pass"]:
        return {"pass": False, "details": {"format": fmt},
                "reason": "FORMAT FAIL: " + "; ".join(fmt["violations"])
                   }

    points = _solution_points(solution)
    ...
    collision_free = polyline_is_collision_free(
        path_xy_enu,
        manifest.obstacles,
        inflation_m=problem.min_clearance_m,
        sample_spacing_m=DEFAULT_SAMPLE_SPACING_M,
    )
    ...
```

Hand-crafted test cases generator snippet:

```python
def _manual_pass_case(problem_id: str) -> WaypointSolution:
    cases: dict[str, list[tuple[float, float]]] = {
        "P1": [(0.8, 0.05), (1.7, 0.10), (2.6, 0.18), (3.5, 0.25), (4.4,
            0.32)],
        "P2": [
            (0.7677425758709568, -0.4696502268112924),
            (1.5348346788044571, -0.9403603783790986),
            (2.296578584877827, -1.41966348208806),
            (3.0978228089887185, -1.8209798385919425),
```

```
            (3.984519252916508, -1.8157225600378732),
        ],
        "P3": [(0.7, 0.40), (1.4, 0.90), (2.1, 1.30), (2.9, 1.60), (3.8,
            1.70)],
        "P4": [
            (0.6761315597043255, 0.5940085080051075),
            (1.3523305204359974, 1.187940291761906),
            (2.0273448194638934, 1.7832159969328145),
            (2.698239599144272, 2.383130323801928),
            (3.4164096427972304, 2.9228814585938196),
        ],
        "P5": [(0.7, -0.10), (1.4, -0.40), (2.2, -0.80), (3.1, -1.10),
            (4.0, -1.20)],
    }
    return WaypointSolution(
        waypoints=[Waypoint(x=x, y=y) for x, y in cases[problem_id]],
        selected_waypoint_index=0,
        reasoning=f"Hand-crafted pass case for {problem_id}.",
    )


def _manual_fail_case(problem_id: str) -> WaypointSolution:
    problem = get_problem(problem_id)
    obstacle = problem.obstacles[0]
    fail_points = [
        Waypoint(x=float(obstacle.center_ned[0]), y=float(obstacle.
            center_ned[1])),
        Waypoint(x=float(obstacle.center_ned[0]), y=float(obstacle.
            center_ned[1])),
        Waypoint(x=float(obstacle.center_ned[0]) - 0.1, y=float(obstacle
            .center_ned[1])),
        Waypoint(x=float(problem.current_position_ned[0]) - 0.4, y=float
            (problem.current_position_ned[1])),
        Waypoint(x=float(problem.current_position_ned[0]) - 0.8, y=float
            (problem.current_position_ned[1])),
    ]
    return WaypointSolution(
        waypoints=fail_points,
        selected_waypoint_index=0,
        reasoning=f"Hand-crafted fail case for {problem_id}.",
    )
```

Manual Verification Test Results:

- **P1** — Pass case: PASS. A hand-crafted nearly straight trajectory satisfies all format, kinematic, progress, and safety checks.

  Fail case: FAIL. The failing example jumps directly onto the obstacle, contains zero-length repeats, and ends with reverse progress.

- **P2** — Pass case: PASS. The hand-crafted pass case uses a clean lower-side go-around with adequate clearance from the center blocker and side guards.

Fail case: FAIL. The failing example drives directly into the center cylinder and also violates the locality and spacing constraints.

- **P3** — Pass case: PASS. The hand-crafted pass case sets up the early slalom direction and maintains the required 0.85 m clearance.

  Fail case: FAIL. The failing example makes huge jumps into the wall/slalom region and is rejected by both the kinematic and safety checks.

- **P4** — Pass case: PASS. The pass case commits upward early and starts escaping the U-shaped trap.

  Fail case: FAIL. The failing example aims straight into the blocked center structure and has excessive step lengths.

- **P5** — Pass case: PASS. The pass case follows a smooth locally feasible corridor line that remains collision-free in the Assignment 2 style scenario.

  Fail case: FAIL. The failing example has grossly infeasible jumps, zero selected progress, and direct collision with the corridor wall/obstacle set.

3 Baseline Evaluation (No Tools) [10 pts]

Instructions: Using the same LLM and `instructor`-based pipeline style from Assignment 2 (no tools), run five independent trials on each of the five problems.

The submission script is `baseline.py`.:

```
python3 baseline.py -mode mock -output-json /tmp/assign3_baseline_mock.json
```

Problem	Difficulty	T1	T2	T3	T4	T5	Pass Rate
P1	Easy	P	P	F	P	P	4/5
P2	Easy	P	F	P	P	F	3/5
P3	Hard	F	P	F	F	F	1/5
P4	Hard	F	F	P	F	F	1/5
P5	Hard	F	F	F	F	F	0/5

Table 1: Baseline results (no tools). P = pass, F = fail.

This trend matches the assignment target well. The two easy cases pass frequently, the two intermediate hard problems only pass once each, and the final Assignment 2 style problem fails in all baseline trials.

4 Tool-Augmented Pipeline [30 pts]

This is the most technically demanding part of the assignment.

Selected Option: A

Tool Description(s):

I implemented a single helper tool in `hybrid_astar_mpc_tool.py`. The tool returns a fully structured `WaypointSolution`, so its I/O is already aligned with the verifier. The intent was to give the model access to exact geometry reasoning and route optimization rather than forcing it to infer a safe path from text alone.

The live prompt flow is now explicit in code and on disk:

- `fixed_prompt.txt`: global task description, schema, and output rules.
- `problem_prompts/P1.txt` ... `problem_prompts/P5.txt`: per-problem state, obstacle geometry, and local success criteria.
- The live pipeline sends the fixed prompt as the system message and the problem prompt as the user message, then appends tool output and verifier feedback when applicable.

Although the assignment only required one helper function for Option A, I made that helper itself internally combine two planning stages:

1. a heading-aware hybrid-A* style graph search to obtain a collision-free coarse route,
2. a smoothing/optimization stage that reuses the repository's existing global path optimizer from `llm_drone.llm.offline_ground_truth_support`.

Why this tool is useful for this domain. These UAV problems fail for two main reasons in the baseline setting: geometry mistakes and local spacing mistakes. The tool directly solves both. The search stage handles obstacle topology, while the optimizer and resampling stage produce smooth local waypoints that satisfy the verifier's kinematic structure much more reliably.

Tool code snippet:

```
def plan_problem(problem_id: str) -> dict[str, Any]:
    problem = get_problem(problem_id)
    manifest = problem.manifest()

    hybrid_path_enu = _hybrid_astar_path(problem.problem_id)
    simplified_enu = simplify_polyline(
        hybrid_path_enu,
        manifest.obstacles,
        inflation_m=problem.min_clearance_m,
        sample_spacing_m=0.2,
    )
    dense_enu = resample_polyline_for_dynamics(
        simplified_enu,
        dt_s=0.5,
```

```
        route_step_m=0.6,
        max_speed_mps=problem.max_speed_mps,
    )
    optimized_enu, optimization_metadata =
        _solve_global_reference_path_xy(
        dense_enu,
        manifest.obstacles,
        planner_clearance_m=problem.min_clearance_m,
        dt_s=0.5,
        max_speed_mps=problem.max_speed_mps,
        max_accel_mps2=problem.max_accel_mps2,
        grid_resolution_m=0.25,
    )
```

Worked Example:

I ran a live smoke test on P1 using:

```
python3 tool_pipeline.py -mode live -problem-id P1
```

Prompt excerpt:

```
Problem P1 (Easy)
Title: Clear Corridor Cruise
- Current position NED is (0.00, 0.00, -2.50) m.
- Goal position NED is (8.00, 0.50, -2.50) m.
- Maintain at least 0.80 m clearance from every listed obstacle.
- north_tree: cylinder centered at NED (4.30, 2.70) m with radius 0.70 m
    .
```

Tool result returned to the pipeline:

```
{
  "waypoints": [
    {"x": 0.898, "y": 0.056},
    {"x": 1.796, "y": 0.112},
    {"x": 2.695, "y": 0.168},
    {"x": 3.593, "y": 0.225},
    {"x": 4.491, "y": 0.281}
  ],
  "selected_waypoint_index": 0,
  "reasoning": "Hybrid A* found a collision-free corridor and the
     optimizer smoothed it with status optimal."
}
```

Final answer: In the live test, the model copied the tool output exactly. That means the run was a genuine LLM API query, but the returned structured answer matched the planner tool's candidate solution.

Verifier output: PASS. The returned trajectory satisfied all format, kinematic, goal-progress, and safety checks.

5 Tool-Augmented Evaluation [10 pts]

Instructions: Re-run the same five-trial evaluation from Section 3, but now using the tool-augmented pipeline.

The submission script is `tool_eval.py`. I generated the following table with:

```
python3 tool_eval.py -mode mock -output-json /tmp/assign3_tool_mock.json
```

Problem	Difficulty	T1	T2	T3	T4	T5	Pass Rate
P1	Easy	P	P	P	P	P	5/5
P2	Easy	P	P	P	P	P	5/5
P3	Hard	P	P	P	P	P	5/5
P4	Hard	P	P	P	P	P	5/5
P5	Hard	P	P	P	P	P	5/5

Table 2: Tool-augmented results. P = pass, F = fail.

The tool removes the dominant error modes from the baseline condition. In the baseline setting, the model often produced overlong waypoint jumps and unsafe direct paths. In the tool setting, the route already comes from deterministic search plus optimization, so the resulting local horizon is both feasible and collision-aware.

6 Self-Refinement with Tools [10 pts]

Instructions: Combine the self-refinement loop from Assignment 2 with the tool-augmented pipeline. Run this on the hardest problem (P5) five times.

The submission script is `refinement.py`. I generated the following table with:

```
python3 refinement.py -mode mock -problem-id P5 -output-json
/tmp/assign3_refine_mock.json
```

Trial	Turn 1	Turn 2	Turn 3	Final
1	P	–	–	P
2	P	–	–	P
3	P	–	–	P
4	P	–	–	P
5	P	–	–	P

Table 3: Self-refinement + tools on P5.

Brief Observations (2–3 sentences): The combination of tools and refinement did outperform the baseline condition on P5, but in this specific implementation refinement did not increase the pass rate beyond the tool-only condition. The reason is simple: once the planning tool is used, the first-turn answer is already verifier-safe, so there is no failure signal for the later refinement turns to correct.

Submission Checklist

Please confirm that your submission includes all of the following:

[x] `SETUP.md` — environment and dependency documentation.

[x] Completed PDF/LaTeX files.

[x] `verifier.py` — verifiers for all five problems.

[x] `baseline.py` — five-trial baseline evaluation.

[x] `tool_pipeline.py` — Option A tool-augmented pipeline.

[x] `tool_eval.py` — five-trial tool-augmented evaluation.

[x] `refinement.py` — refinement + tools loop.

API keys do not appear in any submitted file. The code loads them through environment variables.